Machine Learning

Módulo 3

Optimización

Datasets desbalanceados

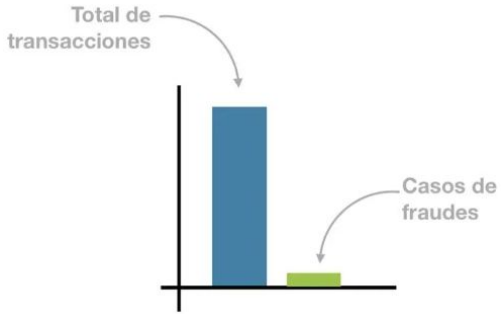
Un dataset **balanceado** es aquel que tiene - aproximadamente - la misma proporción de instancias de cada clase. Por ejemplo, en el caso binario, alrededor de 50:50 (1:1) de cada clase.

Un dataset **desbalanceado** es aquel que tiene muchas instancias de una clase y muy pocas de la otra, dificultando el entrenamiento. Por ejemplo, en el caso binario, 90:10, 99:1, y peor.

La realidad es que un poco de desbalance de clases es de esperar y no afecta a nuestro análisis.

Pero bajo ciertas problemáticas, suelen haber datasets muy desbalanceados:

- Detección de fraudes.
- Diagnóstico médico.
- Falla en cadena de producción.

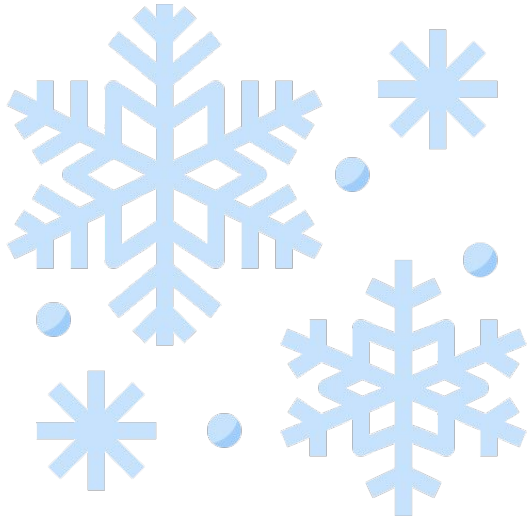


Datasets desbalanceados

Ante un dataset desbalanceado, hay que tener cuidado especialmente con:

- Cómo se entrenan los modelos.
- Qué métricas se usan para evaluarlos.

Existe un concepto denominado **la paradoja de la exactitud** que consiste en que dada una situación, donde la ocurrencia de un evento es de muy baja probabilidad, asumir siempre que no va a pasar va a dar una exactitud muy alta, aunque en sí mismo el modelo sea malo. Por ejemplo, asumir que la caída de nieve en Buenos Aires en cualquier momento, simplemente no va a ocurrir.



Datasets desbalanceados

Ante un dataset con esta característica se pueden tomar diversas acciones:

1. Revisar la posibilidad de conseguir nuevos datos.
2. Utilizar otra métrica que no sea la exactitud. Utilizar **Matriz de Confusión, Precisión y Exhaustividad** (recall) suelen ser las primeras opciones, pero hay más cosas por evaluar. *¿Un Falso Positivo tiene el mismo costo que un Falso Negativo?*
3. Hacerle al dataset un **remuestreo**:
   > Oversampling: generar nuevas instancias de la clase minoritaria, ya sea copiando instancias preexistentes, o generando instancias sintéticas.

   > Undersampling: eliminar instancias de la clase sobrerrepresentada.
4. Probar diferentes modelos (modelos de ensamble suelen ser buenos) y/o agregarle peso a la clase subrepresentada (fácil desde Scikit-Learn).

Datasets desbalanceados

La **exactitud** del modelo es básicamente el número total de predicciones correctas dividido por el número total de predicciones.

La **precisión** de una clase define qué tan confiable es el resultado cuando el modelo responde que un punto pertenece a esa clase.

La **exhaustividad (recall)** de una clase expresa qué tan bien el modelo es capaz de detectarla.

La **puntuación F1** de una clase viene dada por la media armónica de precisión y recuperación, combina la precisión y la exhaustividad (recall) de una clase en una métrica.

Para una clase dada, las diferentes combinaciones de exhaustividad (recall) y precisión tienen los siguientes significados:

- alta exhaustividad + alta precisión: el modelo maneja perfectamente la clase.
- baja exhaustividad + alta precisión: el modelo no puede detectar bien la clase pero es muy confiable cuando lo hace.
- alta exhaustividad + baja precisión: la clase está bien detectada pero el modelo también incluye puntos de otras clases.
- baja exhaustividad + baja precisión: el modelo maneja mal la clase.

Matriz de confusión

Un examen médico tiene una probabilidad de detección de 0.99 y una probabilidad de Falso Positivo de 0.01. El objetivo del Test es detectar una enfermedad de relativa baja prevalencia, que solo la tiene una persona en mil.

Luego de hacer el examen a 100000 personas y completar la matriz de confusión (es decir, calcular, en promedio, cuántos aciertos esperan obtener, cuántos Falsos Negativos, FP y Verdaderos Negativos).

¿Cuál es la probabilidad de que una persona tenga la enfermedad si el examen dio positivo?

Predicha / Verdadera	Positivos	Negativos
Positivos	99	999
Negativos	1	98901

Matriz de confusión

Predicha / Verdadera	Positivos	Negativos
Positivos	Verdaderos Positivos (TP)	Falsos Positivos (FP)
Negativos	Falsos Negativos (FN)	Verdaderos Negativos (TN)

Exactitud = TP + TN / (TP + TN + FP + FN)

Precisión = TP / (TP + FP)

Exhaustividad = TP / (TP + FN)

F1-Score = 2 * Precisión * Exhaustividad / (Precisión + Exhaustividad)

Matriz de confusión

¿Cuál es la probabilidad de que una persona tenga la enfermedad si el examen dio positivo?

Predicha / Verdadera	Positivos	Negativos
Positivos	99	999
Negativos	1	98901

De 1098 predichos, 99 eran verdaderos positivos.
Es decir, ~9% (o probabilidad = 0.0902).

Esa es la **precisión** de la clase.

De 100 realmente positivos, 99 fueron predichos.
Es decir 99%

Esa es la **exhaustividad** de la clase

Teorema de Bayes

¿Cuál es la probabilidad de que una persona tenga la enfermedad si el examen dio positivo?
Usamos el **Teorema de Bayes**:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

P(A|B): posterior o probabilidad a posteriori

P(E+|T+): probabilidad de estar enfermo dado que el test dio positivo.

0.99*0.001/0.01098 = 0.09

P(B|A): verosimilitud

P(T+|E+): probabilidad de que el test de positivo dado que la persona está enferma. ¡Es la probabilidad de detección! **= 0.99**

P(A): prior o probabilidad a priori de A

P(E+): El prior es la prevalencia de la enfermedad en la población **= 1/1000**

P(B): probabilidad marginal.

P(T+). La probabilidad de que el test dé positivo. Esto puede ocurrir si una persona está enferma pero también si no lo está. **= P(T+|E+)*P(E+) + P(T+|E-)*P(E-) = 99/100*0.001 + 999/99900*0.999 = 0.01098**

A: estar enfermo	**E+**
B: dio positivo	**T+**

Enlace recomendado: The Bayesian Trap

Teorema de Bayes

- El **Teorema de Bayes** tiene en cuenta automáticamente la prevalencia de las clases.
- Dada una clasificación Binaria entre C+ y C-, llamamos X a los atributos, la formulación más general del problema de clasificación es:

$$P(C+|X) = P(X|C+)\,P(C+) / P(X) \text{ y } P(C-|X) = P(X|C-)\,P(C-) / P(X)$$

- En general, es muy difícil formular de manera completa este problema. Necesitaríamos saber de qué tipo de distribución tiene cada feature, cómo están correlacionados, etc

Teorema de Bayes

Dados dos eventos A y B:

- P(A) es la probabilidad del evento A
- P(B) es la probabilidad del evento B
- P(A|B) es la probabilidad condicional del evento A dado que ocurrió B
- P(B|A) es la probabilidad condicional del evento B dado que ocurrió A
- Si P(A|B) = P(A) y P(B|A) = P(B), los eventos son independientes.

} No implica causalidad.

En general, **P(A|B) ≠ P(B|A).**
Para obtener uno dado el otro, necesitamos el Teorema de Bayes:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Teorema de Bayes

- Explica como uno debe cambiar sus creencias, teniendo en cuenta nueva evidencia.
- Con la probabilidad total a partir de las probabilidades del suceso A (probabilidad de que llueva o de que haga buen tiempo) deducimos la probabilidad del suceso B (que ocurra un accidente).
- Con el teorema de Bayes, a partir de que ha ocurrido el suceso B, deducimos las probabilidades del suceso A.



Naive Bayes

Este modelo está basado en el Teorema de Bayes con un supuesto de independencia entre los predictores. **Naive Bayes** supone que la presencia de una característica particular en una clase no está relacionada con la presencia de ninguna otra característica.

Por ejemplo, una fruta puede considerarse una manzana si es roja, redonda y de aprox. 3” de diámetro. Incluso si estas características dependen unas de otras o de la existencia de otras características, todas estas propiedades contribuyen independientemente a la probabilidad de que esta fruta sea una manzana y por eso se conoce como "ingenua".

El modelo Naive Bayes es fácil de construir y particularmente útil para conjuntos de datos muy grandes.

Naive Bayes

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Tomando como ejemplo un conjunto de datos sobre el clima y la variable "Jugar". Debemos clasificar si los jugadores jugarán o no según las condiciones climáticas.

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

Naive Bayes

Tenemos que la probabilidad de que esté nublado es de 0.29, mientras que la probabilidad de jugar es de 0.64.

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

La pregunta que nos hacemos es:
¿Los jugadores jugarán si hay sol?

A continuación, se usa la ecuación Bayesiana para calcular la probabilidad a posteriori para cada clase. La clase con la probabilidad a posteriori más alta es el resultado de la predicción.

Naive Bayes

P(Yes | Sunny) = P(Sunny | Yes) * P(Yes) / P (Sunny)

P (Sunny |Yes) = 3/9 = 0.33
P(Sunny) = 5/14 = 0.36
P(Yes)= 9/14 = 0.64

P (Yes | Sunny) = 0.33 * 0.64 / 0.36 = 0.60

Nos queda que 0.6 es la probabilidad de que jueguen, si hay sol.

Naive Bayes

Ventajas

- Es fácil de entender y rápido.
- Funciona bien en la predicción de clases múltiples.
- Cuando se asume la independencia, un clasificador Naive Bayes funciona mejor en comparación con otros modelos como la regresión logística y necesita menos datos de entrenamiento.
- Funciona bien en el caso de variables de entrada categóricas.

Desventajas

- Suposición de variables independientes. En la vida real es casi imposible que obtengamos un conjunto de variables completamente independientes.
- Para las variables numéricas, supone una distribución Normal o Gaussiana, lo que implica un supuesto fuerte.

Naive Bayes Multidimensional

- Ejemplo para 4 atributos:

$$P(A|b_1,b_2,b_3,b_4)=P(A)\cdot(P(b_1|A)\cdot P(b_2|A)\cdot P(b_3|A)\cdot P(b_4|A))$$

- Para n atributos:

$$P(A|b_1,b_2,\dots,b_n)=P(A)\cdot P(b_1,b_2,\dots,b_n|A)=P(A)\cdot\prod_{i=1}^{n}P(a_i|A)$$

- Fórmula general de Naive Bayes:

$$Solucion=arg\max_{i=1}^{n}P(c_i)\cdot\prod_{j=1}^{m}P(a_j|c_i)$$

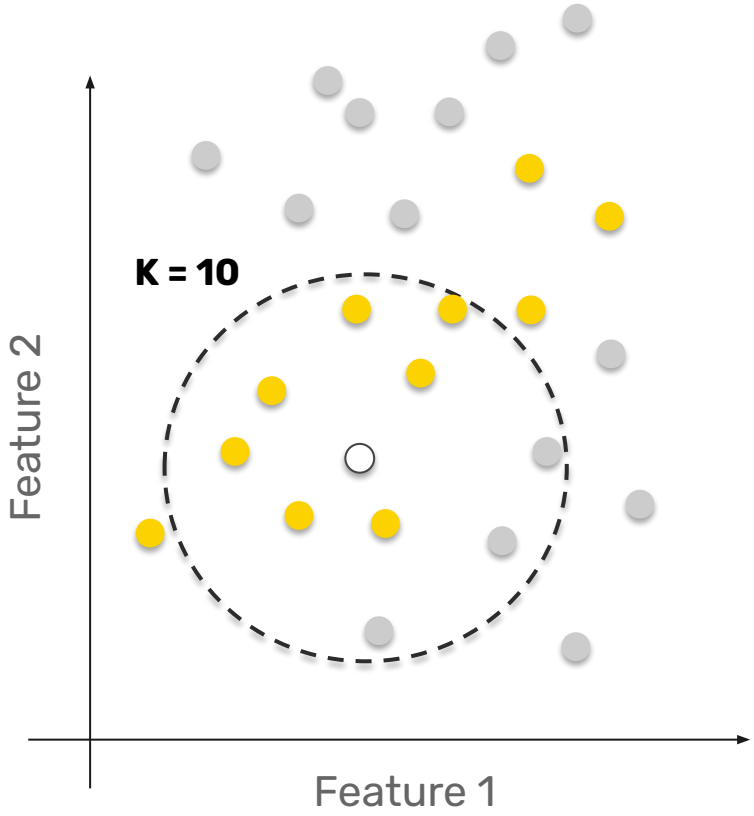
Scores

La Matriz de Confusión tiene toda la información respecto del desempeño del algoritmo en relación a los casos reales, pero... ¿cómo se llega a la decisión de etiquetar en cada caso? ¿qué grado de certeza tiene el algoritmo?

Por ejemplo, dado un modelo de vecinos más cercanos, con K = 10.

En este caso, de 10 vecinos, 7 son amarillos, por lo que la etiqueta correspondiente sería amarilla.

Lo mismo ocurrirá cuando haya más de 5 vecinos de color amarillo. Si, en cambio, hay menos de 5 vecinos de color amarillo, la etiqueta pasa a ser gris.

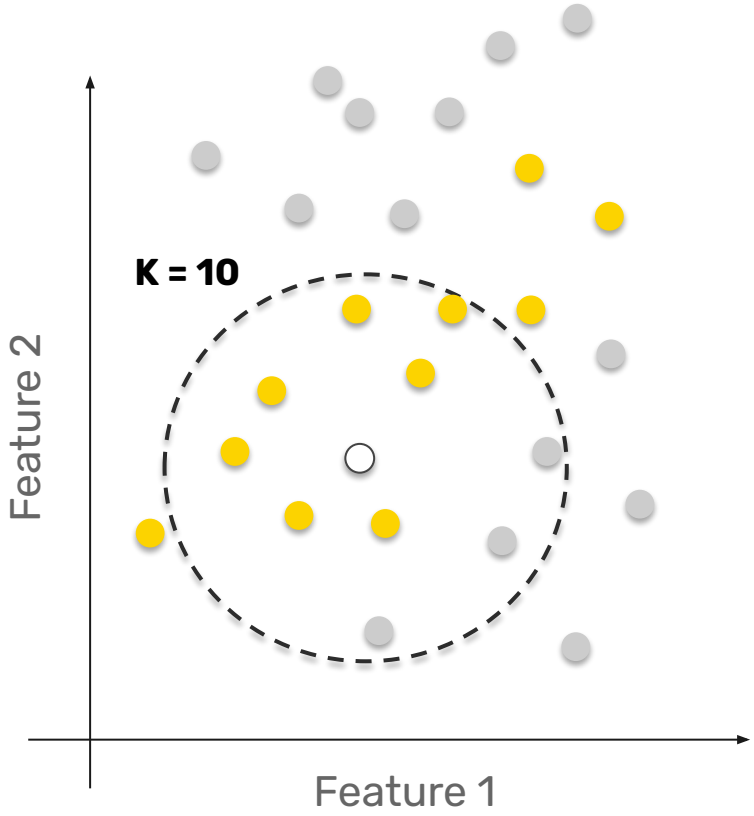


Scores

Entonces, de una instancia que tiene 10 vecinos amarillos, podemos estar más seguros que la etiqueta correspondiente es amarrilla que una instancia que solamente tiene 6 vecinos.

Cuando miramos solamente las etiquetas asignadas, esta información la perdemos.

Sin embargo, es posible generar un Score que represente cuán seguro está el modelo de la etiqueta.



Scores

Este razonamiento se puede hacer con (casi) todos los modelos que usemos. En el fondo, lo que un modelo hace para asignar etiquetas es generar un score y poner el umbral a la mitad.

En Scikit-Learn, todos los modelos vienen con un método, **predict_proba(X)**, que calcula los scores.

Si bien a primer orden estos scores pueden ser interpretados como probabilidades, la realidad es que no lo son, porque no están **Calibrados**. Lo que sí podemos usar son los **Rankings** que generan.

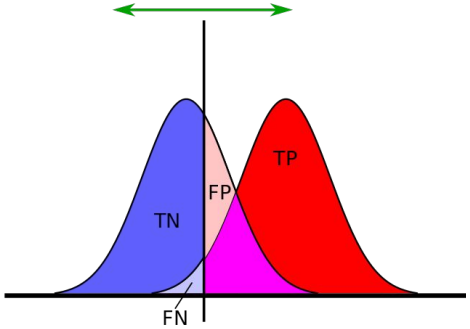


Curva ROC

La decisión de clasificar un elemento es parte del algoritmo, esa elección puede estar fundada bajo diferentes niveles de certidumbre, pero ¿se podría obtener otros resultados si modificamos el **umbral** mediante el cual el algoritmo decide para clasificar?

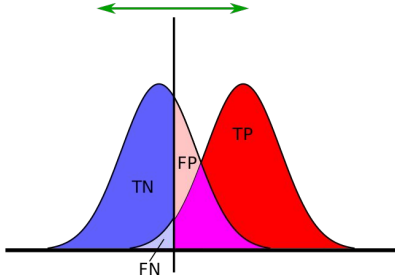
Continuando con el ejemplo del test de enfermedad, se puede decir que en función del umbral habrá una variación de exhaustividad. Si el umbral es alto, la exhaustividad disminuye, porque se es muy "estricto" a la hora de identificar un caso positivo. Por el contrario, si se disminuye, la exhaustividad aumenta, porque el criterio para definir un positivo es muy "laxo".

Es importante notar que a medida que se mueve el umbral, cambia la cantidad de Verdaderos positivos, Falsos positivos, Falsos Negativos y Verdaderos Negativos.



TP	FP
FN	TN

Curva ROC



TP	FP
FN	TN

Para cuantificar esa variación, se establece:

$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}$$

True Positive Rate → Es la **Exhaustividad**

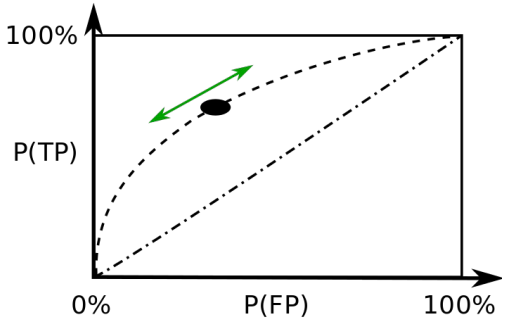
$$FPR = \frac{FP}{N} = \frac{FP}{FP + TN}$$

False Positive Rate → Es la **Especificidad**

La Curva ROC es un gráfico que muestra esa relación y representa el rendimiento de un modelo de clasificación en todos los umbrales.

- TPR: Tasa de Verdaderos Positivos
- FPR: Tasa de Falsos Positivos

Reducir el umbral de clasificación clasifica más elementos como positivos, por lo que aumentarán tanto los Falsos positivos como los Verdaderos positivos.



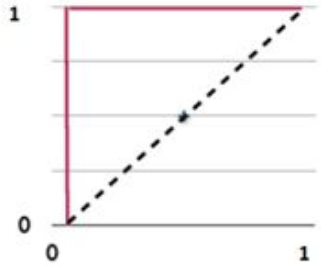
Curva ROC

Debido a que la curva ROC es difícil de calcular, a menudo es más habitual trabajar con la **AUC**, **área bajo la curva**, que da una medición agregada del rendimiento en todos los umbrales de clasificación posibles.

Una forma de interpretar el AUC es como la probabilidad de que el modelo clasifique un ejemplo positivo aleatorio más alto que un ejemplo negativo aleatorio.

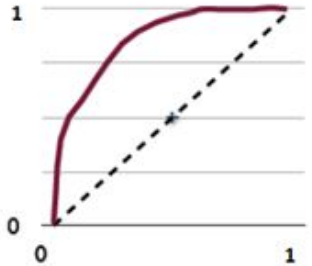
AUC = 1

+ valor diagnóstico perfecto



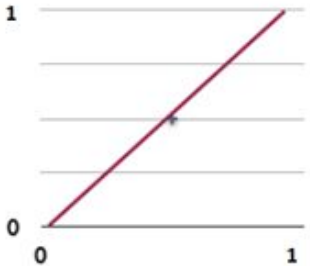
AUC = 0.8

+ valor diagnóstico



AUC = 0.5

+ sin valor diagnóstico

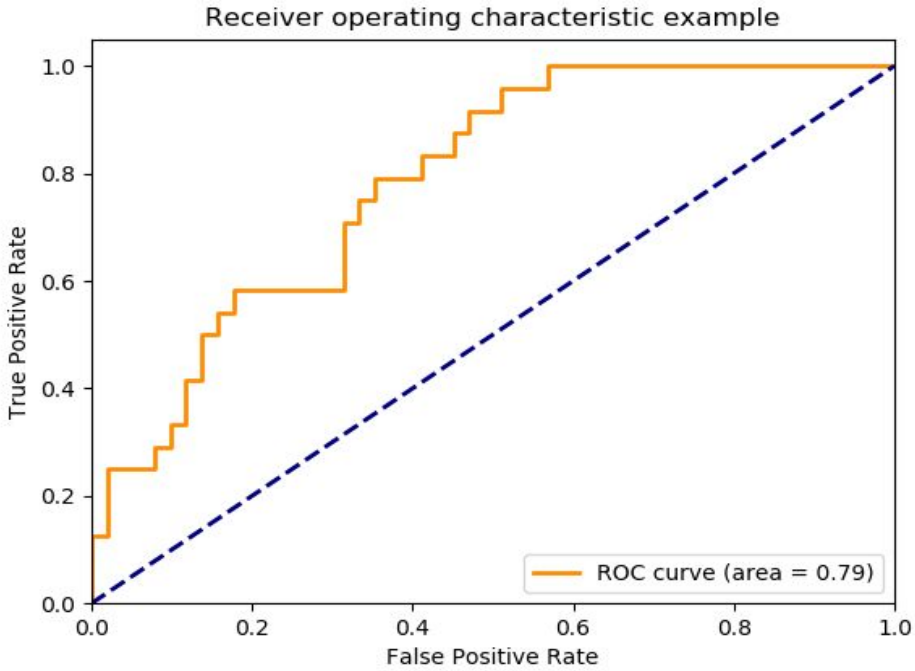


Curva ROC

¿Qué valor de umbral es mejor?

Esto va a depender del problema, si se quiere favorecer precisión o exhaustividad.

Pero al área bajo la curva es un indicador de cuán bueno es nuestro modelo independientemente de eso.



Curva ROC

- **Una curva ROC *(Receiver Operating Characteristic)* es una representación gráfica que ilustra la relación entre TPR y el FPR** de un sistema clasificador para diferentes puntos de corte. NO confundir con una curva Precision-Recall.
- Se puede usar **para generar estadísticos que resumen el rendimiento o la efectividad de un clasificador**. El indicador más utilizado en muchos contextos es el área bajo la curva ROC o AUC (AUC-*Área Bajo la Curva*).

- Dato histórico: fue desarrollada por ingenieros eléctricos en la 2º Guerra Mundial, para medir la eficacia de la detección de objetos enemigos en el campo de batalla mediante señales de radar. Su uso está muy extendido en medicina para validar técnicas diagnósticas.

Enlace recomendado: https://www.bioestadistica.uma.es/analisis/roc1

Curva ROC

Implementación en Scikit-Learn:

```
from sklearn.metrics import roc_curve
y_pred_rt = clf_2_knn.predict_proba(X_no_medido)[:, 1]

fpr_rt_lm, tpr_rt_lm, _ = roc_curve(y_no_medido, y_pred_rt)
fpr_rt_lm
```

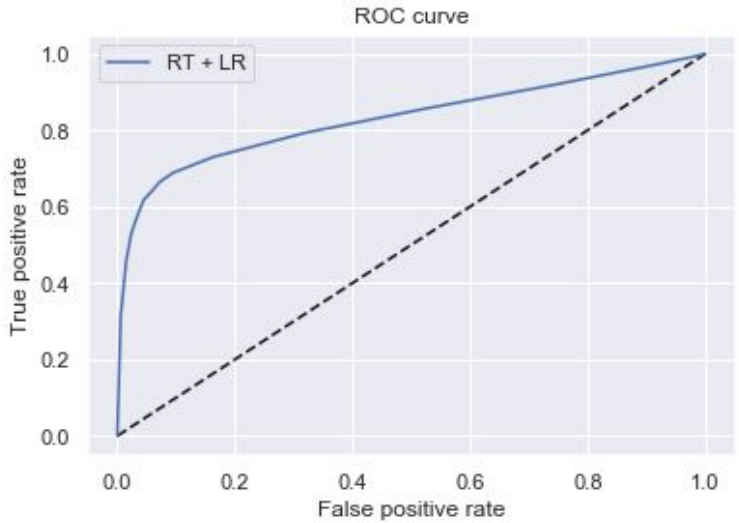
```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_no_medido, y_pred_rt)
```

```
0.8305866875863983
```

```
from sklearn import metrics
metrics.auc(fpr_rt_lm, tpr_rt_lm)
```

```
0.8305866875863983
```

```
plt.figure(1)
plt.plot([0, 1], [0, 1], 'k--')
plt.plot(fpr_rt_lm, tpr_rt_lm, label='RT + LR')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve')
plt.legend(loc='best')
plt.show()
```

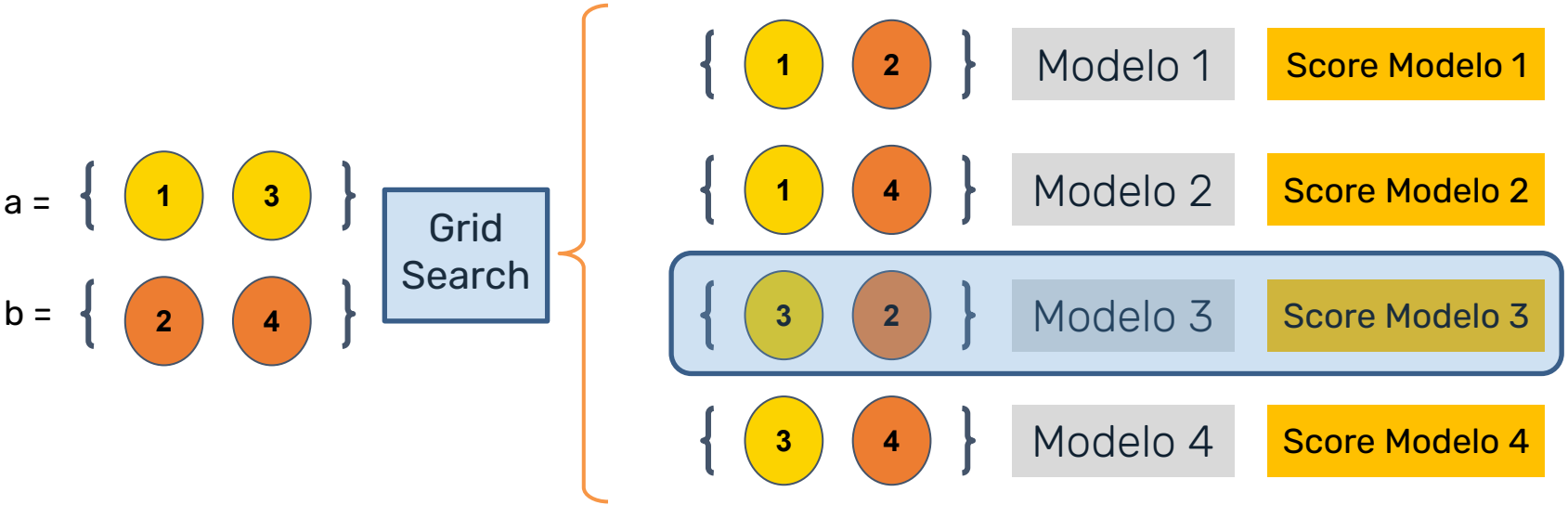


Optimización de hiperparámetros

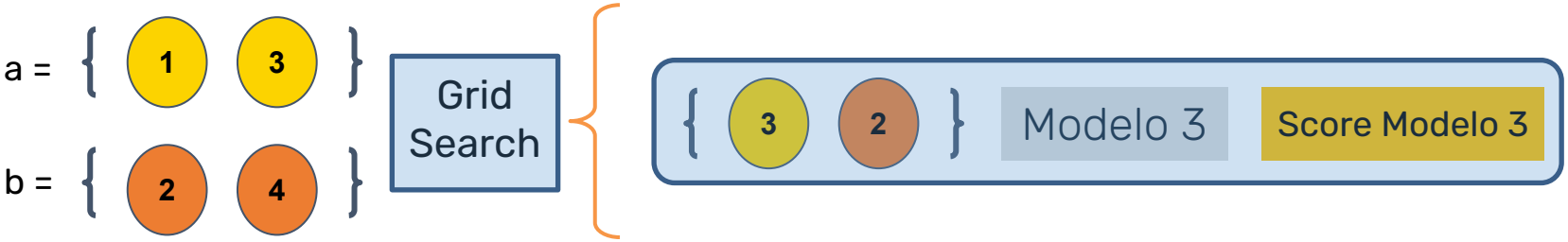
- ¿Cómo elegimos los mejores hiperparámetros para nuestro problema?
- ¿Qué es mejor, exactitud, precisión o exhaustividad? ¿Área bajo la curva ROC?
- Primero, se debe definir una métrica a optimizar. **Una vez que se sabe cuál métrica optimizar,** hay que probar los distintos valores de hiperparámetros.
- Se debe hacer una búsqueda exhaustiva. Es decir probando con todos los valores de los hiperparámetros que podamos y eligiendo la mejor combinación. Este método se llama **Grid Search** (“búsqueda de cuadrícula”).

Optimización de hiperparámetros

Por ejemplo, si tenemos dos hiperparámetros, a y b,
que pueden tomar valores a = {1,2} y b = {3,4}



Optimización de hiperparámetros



Grid Search consiste en:

1. Elegir los valores que puede tomar cada hiperparámetro
2. Armar las combinaciones “todos con todos” → Armar la grilla
3. Recorrer la grilla entrenando el modelo para cada combinación y evaluarlo.
4. Elegir los hiperparámetros que definen el mejor modelo.

Optimización de hiperparámetros

¿Cómo evaluar el modelo?

Al estar probando MUCHOS modelos, podría suceder que uno se desempeñe muy bien en el conjunto de Train simplemente por azar.

Por esto ,es muy importante evaluar cada modelo creado por Grid Search con Validación Cruzada en el conjunto de entrenamiento.

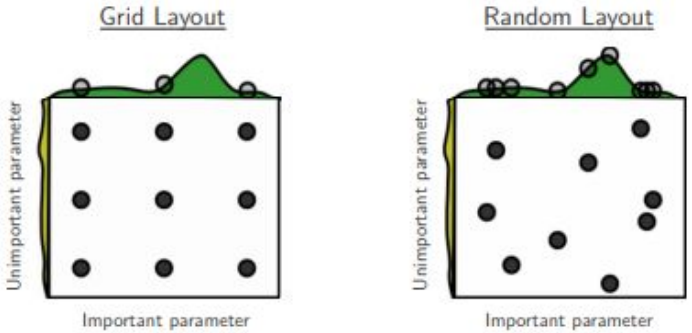
Grid Search y Validación Cruzada suelen venir juntos.



Optimización de hiperparámetros

Random Search

- Si por ejemplo, se tienen cinco hiperparámetros y cinco valores para probar por hiperparámetro, el tamaño de la grilla comienza a crecer.
- Además, para cada modelo se debe hacer la Validación Cruzada.
- Este proceso puede ser computacionalmente muy demandante.
- **Random Search explora opciones y combinaciones al azar**, de manera menos "ordenada". En muchas circunstancias, esto es más eficiente, tanto desde el punto de vista de performance del modelo como de desempeño computacional.



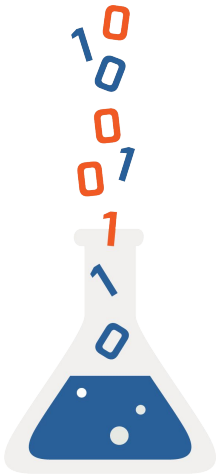
Conclusiones

1. Es necesario definir una métrica a optimizar (exactitud, precisión, RMSE, ROC AUC, etc.)
2. Un modelo (regresor o clasificador)
3. Una grilla de hiperparámetros. Depende del tipo de modelo utilizado.
4. Un método para buscar o muestrear los candidatos
    - Grid Search: Plantea opciones y explora todas las combinaciones
    - Random Search: explora opciones y combinaciones al azar.

¡Importante!

Crear un modelo lo antes posible, en cualquier caso, un modelo fallido muchas veces da tanta información sobre el proceso real como uno válido...





Lab Time!

¡Muchas gracias!

¡Sigamos trabajando!